



Gamified Smart Habit Tracker

Complete Technical Documentation

Version 1.0 | April 2025

Prepared for Development Team Use

11 Tables	40+ Endpoints	7 Services	9 Sections
Database Schema	REST API	Backend Logic	Documentation

1. Executive Summary

1.1 Project Purpose & Goals

The Gamified Smart Habit Tracker is a web application designed to help users build and maintain positive habits through the motivational power of gamification. The platform transforms mundane daily routines into engaging quests, rewarding users with XP, coins, badges, and levels as they progress toward their personal goals.

Core goals of the system include:

- Increase habit retention through intrinsic and extrinsic motivation mechanisms
- Provide measurable progress feedback via streaks, levels, and achievement badges
- Foster healthy social competition through leaderboards and friend challenges
- Sustain long-term engagement via seasonal challenges and reward shops
- Deliver a data-rich platform that enables personalized habit recommendations

1.2 Target Audience

Segment	Characteristics	Use Cases
Individuals	Health-conscious users aged 18–40	Exercise, meditation, reading habits
Students	Goal-oriented, competitive learners	Study schedules, productivity goals
Corporate Teams	Professional development focus	Wellness challenges, team goals
Coaches & Trainers	Client accountability tracking	Program adherence monitoring

1.3 Key Features Overview

Core Habit Engine

- Daily and weekly habit scheduling with configurable difficulty tiers (Easy / Medium / Hard)
- Flexible goal types supporting binary (did/did not) and quantitative (e.g., ran 5 km) logging
- Automatic streak tracking with grace period support

Gamification System

- XP-based progression with a mathematically smooth level formula
- Coin economy for purchasing cosmetic and functional rewards
- Badge achievement system with 5+ criteria types
- Time-limited challenges with bonus XP and badge rewards

Social Features

- Bi-directional friend system with pending/accepted/blocked states
- Global and friends-only leaderboards ranked by XP

- Friend activity feeds and challenge invitations

Reward Shop

- Avatar items, themes, consumables, and exclusive badges
- JSONB metadata for flexible item configurations
- Full purchase history and inventory tracking

Notification System

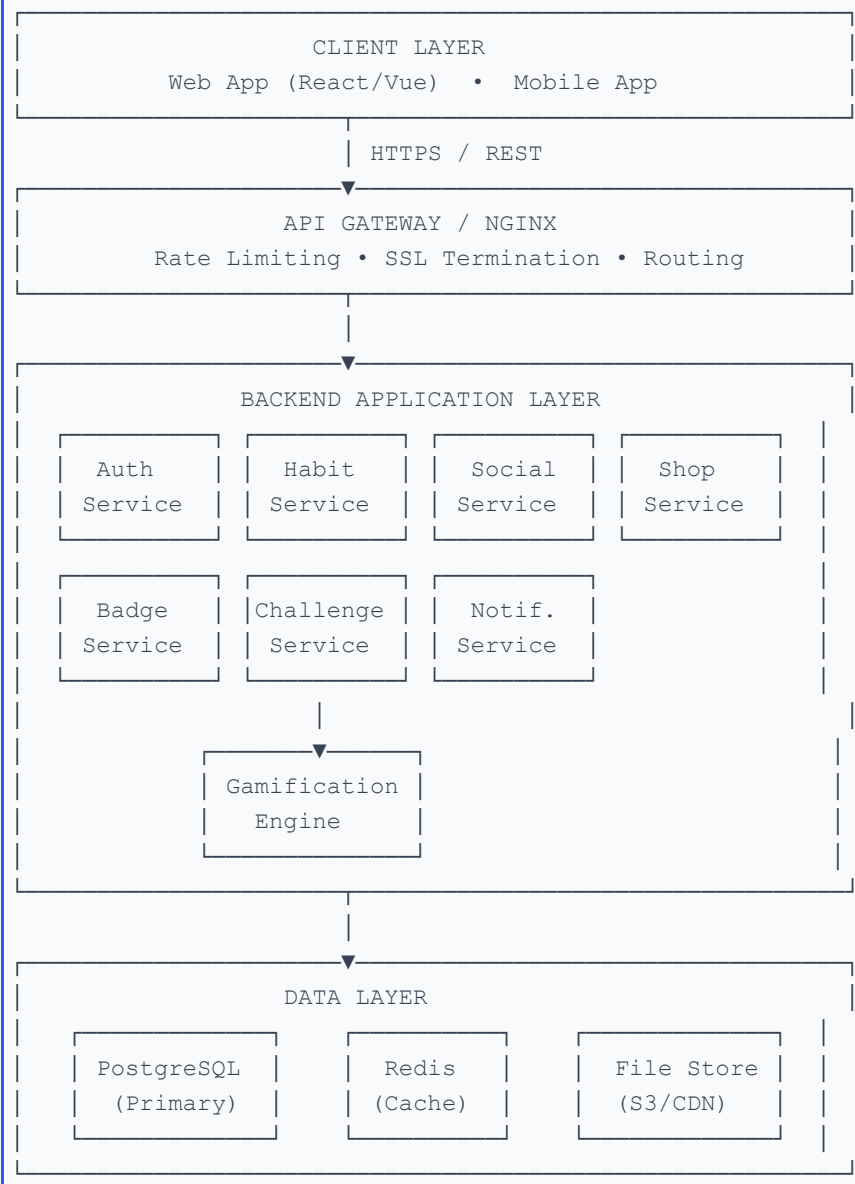
- In-app alerts for level-ups, badge unlocks, streak milestones, and challenge updates
- Extensible type system for push notification integration

2. System Architecture

2.1 High-Level Architecture

The system follows a layered REST API architecture with a stateless backend, relational database, and optional caching layer. All components are containerized for consistent deployment.

Architecture Diagram (text representation):



2.2 Technology Stack Recommendations

Layer	Technology	Rationale
Runtime	Node.js 20 LTS	Non-blocking I/O, rich ecosystem, TypeScript support
Framework	Express.js / Fastify	Lightweight, mature middleware ecosystem
Database	PostgreSQL 16	JSONB support, UUID, soft deletes, full ACID compliance
ORM	Prisma / Knex.js	Type-safe queries, schema migrations, connection pooling
Cache	Redis 7	Leaderboard sorted sets, session storage, rate limiting
Auth	JWT + bcrypt	Stateless authentication, secure password hashing
File Storage	AWS S3 / MinIO	Avatar and badge image hosting with CDN delivery
API Gateway	NGINX	Reverse proxy, SSL, rate limiting, load balancing
Container	Docker + Compose	Consistent dev/prod environments, easy scaling
Scheduler	node-cron / Bull	Streak resets, challenge auto-completion, reminders
Monitoring	Prometheus + Grafana	Metrics, dashboards, alert rules

2.3 Data Flow Description

Habit Completion Flow

1. User submits POST /habits/{id}/log with completion value
2. Auth middleware validates JWT and extracts user_id
3. Habit Service verifies habit ownership and checks daily limit
4. Gamification Engine calculates XP based on difficulty and streak multiplier
5. Streak Tracker updates current_streak and checks for new max_streak
6. Badge Service asynchronously checks all relevant badge criteria
7. Notification Service queues alerts for level-up or badge unlock events
8. Response returns updated stats: XP gained, new level, coins, streak

Authentication Flow

9. Client POSTs credentials to /auth/login
10. Auth Service validates email + bcrypt password comparison
11. JWT signed with RS256, 15-minute access token + 7-day refresh token issued
12. Subsequent requests include Bearer token in Authorization header
13. Middleware verifies signature, checks expiry, attaches user context

3. Database Documentation

3.1 ER Diagram (Mermaid)

Paste the following Mermaid code into <https://mermaid.live> to render the full ER diagram.

```
erDiagram
    users {
        uuid id PK
        string username
        string email
        int xp
        int level
        int coins
        int current_streak
        int max_streak
        string avatar_url
        timestamp created_at
        timestamp deleted_at
    }
    habits {
        uuid id PK
        uuid user_id FK
        string name
        enum goal_type
        enum difficulty
        int target_value
        bool is_active
        timestamp deleted_at
    }
    habit_logs {
        uuid id PK
        uuid habit_id FK
        uuid user_id FK
        date logged_date
        int value
        int xp_earned
        timestamp created_at
    }
    users ||--o{ habits : "owns"
    habits ||--o{ habit_logs : "has"
    users ||--o{ habit_logs : "records"
    users ||--o{ user_badges : "earns"
    badges ||--o{ user_badges : "awarded_as"
    users ||--o{ user_challenges : "participates"
    challenges ||--o{ user_challenges : "contains"
    users ||--o{ purchases : "makes"
```

```
reward_shop ||--o{ purchases : "sold_as"
users ||--o{ friends : "has"
users ||--o{ notifications : "receives"
```

3.2 Table Documentation

users

Stores all registered user accounts, gamification progress, and profile data.

Column	Type	Nullable	Description
id	UUID	No	Primary key, auto-generated via gen_random_uuid()
username	VARCHAR(50)	No	Unique display name, alphanumeric + underscores
email	VARCHAR(255)	No	Unique email, used for login and notifications
xp	INTEGER	No	Total experience points earned (default: 0)
level	INTEGER	No	Derived from XP; computed as floor(sqrt(xp/100))+1
coins	INTEGER	No	Spendable currency for reward shop purchases
current_streak	INTEGER	No	Consecutive days with at least one habit logged
max_streak	INTEGER	No	Historical best streak, never decremented
avatar_url	TEXT	Yes	Full URL to avatar image stored in S3/CDN
created_at	TIMESTAMPTZ	No	Account creation timestamp (UTC)
deleted_at	TIMESTAMPTZ	Yes	Soft delete: non-NULL means account is deactivated

Indexes: email (UNIQUE), username (UNIQUE), deleted_at (partial for active users)

habits

Tracks individual habits created by users, including scheduling type and difficulty settings.

Column	Type	Nullable	Description
id	UUID	No	Primary key
user_id	UUID	No	FK → users.id (CASCADE DELETE)
name	VARCHAR(100)	No	Human-readable habit title
description	TEXT	Yes	Optional detailed description of the habit

goal_type	ENUM('daily','weekly')	No	Determines reset frequency for streak calculations
difficulty	ENUM('easy','medium','hard')	No	Affects XP multiplier (1×, 2×, 3×)
target_value	INTEGER	No	Completion threshold (1 for binary, n for quantitative)
is_active	BOOLEAN	No	Paused habits skip streak checks (default: true)
deleted_at	TIMESTAMPTZ	Yes	Soft delete; preserves historical logs

Indexes: user_id, (user_id, is_active), deleted_at (partial)

habit_logs

Immutable record of each habit completion event. One log per habit per day (enforced by unique constraint).

Column	Type	Nullable	Description
id	UUID	No	Primary key
habit_id	UUID	No	FK → habits.id
user_id	UUID	No	FK → users.id (denormalized for fast lookups)
logged_date	DATE	No	Local calendar date of completion (YYYY-MM-DD)
value	INTEGER	No	Logged quantity; 1 for binary, actual count for quantitative
xp_earned	INTEGER	No	XP awarded at time of logging (immutable snapshot)
created_at	TIMESTAMPTZ	No	Exact server timestamp of submission

Unique Constraint: (habit_id, logged_date) — prevents duplicate entries per day.

Indexes: user_id, habit_id, logged_date, (user_id, logged_date)

badges, user_badges, challenges, user_challenges

Table	Purpose
badges	Catalog of all achievable badges. criteria_type: streak total_xp completions level challenge. criteria_value is the threshold to unlock.
user_badges	Many-to-many join between users and badges. UNIQUE(user_id, badge_id) prevents duplicate awards. earned_at provides audit trail.
challenges	Time-boxed competitive events. start_date and end_date define the active window. Linked to a reward badge_id for completion bonus.

user_challenges	Tracks individual participation: status (joined/completed/failed), numeric progress, and completion timestamp for leaderboard ranking.
-----------------	--

3.3 Cardinality Matrix

From	To	Cardinality	Notes
users	habits	1 : Many	Each user owns multiple habits
habits	habit_logs	1 : Many	Each habit has many logs (max 1/day)
users	badges	Many : Many	Via user_badges join table
users	challenges	Many : Many	Via user_challenges join table
users	reward_shop	Many : Many	Via purchases join table
users	friends	Many : Many	Self-referential with direction and status
users	notifications	1 : Many	Notification inbox per user
challenges	badges	Many : 1	Challenge completion can award a badge

3.4 Sample Queries

Get user dashboard stats

```
SELECT u.username, u.xp, u.level, u.coins, u.current_streak,
       COUNT(ub.id) AS badge_count
FROM users u
LEFT JOIN user_badges ub ON ub.user_id = u.id
WHERE u.id = $1 AND u.deleted_at IS NULL
GROUP BY u.id;
```

Get habits logged today

```
SELECT h.name, hl.value, hl.xp_earned
FROM habits h
JOIN habit_logs hl ON hl.habit_id = h.id
WHERE hl.user_id = $1
      AND hl.logged_date = CURRENT_DATE
      AND h.deleted_at IS NULL;
```

Global leaderboard (top 10)

```
SELECT username, xp, level, current_streak
FROM users
WHERE deleted_at IS NULL
ORDER BY xp DESC
```

```
| LIMIT 10;
```

Check streak candidates for reset (cron job)

```
SELECT DISTINCT u.id, u.current_streak
FROM users u
WHERE u.current_streak > 0
      AND u.deleted_at IS NULL
      AND NOT EXISTS (
        SELECT 1 FROM habit_logs hl
        WHERE hl.user_id = u.id
              AND hl.logged_date = CURRENT_DATE - INTERVAL '1 day'
      );
```

4. Backend API Documentation

| Base URL: <https://api.habittracker.io/v1> | All responses are JSON | Authentication: Bearer <JWT>

4.1 Authentication

POST /auth/register

Register a new user account.

Field	Details
Method	POST
Auth	None (public)
Request Body	{ "username": "john_doe", "email": "john@example.com", "password": "SecurePass123!" }
Success 201	{ "user": { "id": "uuid", "username": "john_doe" }, "token": "eyJ..." }
Errors	400 Validation error 409 Email/username already taken

POST /auth/login

Field	Details
Method	POST
Auth	None (public)
Request Body	{ "email": "john@example.com", "password": "SecurePass123!" }
Success 200	{ "access_token": "eyJ...", "refresh_token": "eyJ...", "expires_in": 900 }
Errors	401 Invalid credentials 423 Account locked

4.2 Users

M	Endpoint	Description
GET	/users/me	Get authenticated user profile with XP, level, coins, streak
GET	/users/me/stats	Aggregated stats: total habits, total logs, XP history by week
PUT	/users/me	Update profile fields: username, avatar_url (others restricted)

4.3 Habits

M	Endpoint	Description
GET	/habits	List all active habits for authenticated user. Query: ?goal_type=daily
POST	/habits	Create new habit. Body: name, goal_type, difficulty, target_value, description
GET	/habits/:id	Get single habit with today's completion status
PUT	/habits/:id	Update habit fields. Cannot change difficulty retroactively
DELETE	/habits/:id	Soft delete habit. Sets deleted_at, preserves logs
POST	/habits/:id/log	Log habit completion. Body: { value: 1 }. Awards XP and coins
GET	/habits/:id/history	Paginated log history. Query: ?from=2025-01-01&to=2025-03-31

Log Habit — Example Response

```
{
  "log": { "id": "uuid", "xp_earned": 30, "logged_date": "2025-04-06" },
  "rewards": {
    "xp_gained": 30,
    "coins_gained": 5,
    "new_level": null,
    "badges_unlocked": ["First Week Warrior"],
    "streak": { "current": 7, "is_new_max": false }
  }
}
```

4.4 Social & Leaderboards

M	Endpoint	Description
POST	/friends/request	Send friend request. Body: { friend_id: "uuid" }
PUT	/friends/:id/respond	Accept or decline request. Body: { action: "accept" "decline" }
DELETE	/friends/:id	Remove or block friend. Body: { action: "remove" "block" }
GET	/leaderboards/global	Top 100 users by XP. Query: ?period=weekly monthly all_time
GET	/leaderboards/friends	Friends-only leaderboard for authenticated user

4.5 Error Response Format

```
{
  "error": {
    "code": "HABIT_ALREADY_LOGGED",
    "message": "This habit has already been logged today.",
  }
}
```

```
"status": 409,  
  "timestamp": "2025-04-06T10:30:00Z"  
}  
}
```

4.6 Rate Limiting Strategy

Route Group	Limit	Window	Header
/auth/*	5 requests	15 minutes	X-RateLimit-*
/habits/:id/log	1 request	Per day per habit	Business rule
General API	100 requests	1 minute	X-RateLimit-*
/leaderboards/*	30 requests	1 minute	Cached 60s in Redis

5. Business Logic Documentation

5.1 XP Calculation Formula

$$XP\text{ Earned} = Base\text{ XP} \times Difficulty\text{ Multiplier} \times Streak\text{ Bonus Multiplier}$$

Parameter	Value	Notes
Base XP (Easy)	10 XP	Lowest entry point to reward all completion attempts
Base XP (Medium)	20 XP	2× Easy; reflects meaningful effort
Base XP (Hard)	30 XP	3× Easy; reserved for challenging commitments
Streak Bonus (7+ days)	×1.25	25% bonus for maintaining a week-long streak
Streak Bonus (30+ days)	×1.5	50% bonus for month-long dedication
Streak Bonus (100+ days)	×2.0	Double XP for elite streaks

XP Calculation Example

```
// Hard habit, 30-day streak
base_xp = 30
streak_multiplier = 1.5    // 30+ days
xp_earned = Math.floor(30 * 1.5)  // = 45 XP

// Coin reward: floor(xp_earned / 10)
coins_earned = Math.floor(45 / 10)  // = 4 coins
```

5.2 Level Formula

$$level = floor(sqrt(xp / 100)) + 1$$

Level	XP Required	XP to Next	Notes
1	0	100	Starting level for all new users
2	100	300	First level-up after ~10 hard habits
5	1,600	900	Represents 1–2 weeks of daily activity
10	8,100	1,900	Approximately 1 month of consistent effort
20	36,100	3,900	Long-term committed users (3–6 months)
50	240,100	9,900	Elite status — 1+ year of daily use

```
// Level calculation in code
function calculateLevel(xp) {
  return Math.floor(Math.sqrt(xp / 100)) + 1;
}

// XP needed to reach a specific level
function xpForLevel(level) {
  return Math.pow(level - 1, 2) * 100;
}
```

5.3 Streak Logic

The streak represents consecutive calendar days on which the user logged at least one habit completion.

Increment Rules

- Streak increments by 1 when the user logs any habit on the current calendar day
- If yesterday already had a log, the streak continues; otherwise it starts at 1
- max_streak is updated only when current_streak exceeds it

Reset Rules

- A scheduled cron job runs at 00:05 UTC daily to identify users who had no logs yesterday
- Those users have current_streak reset to 0 (not decremented — hard reset)
- A "streak broken" notification is queued for affected users

Grace Period (optional)

Implementation Note: A purchasable consumable "Streak Shield" in the reward shop can protect a streak for one missed day. This is stored as meta_data on the user record or as an active consumable purchase.

5.4 Badge Criteria Algorithm

```
async function checkAndAwardBadges(userId, eventContext) {
  const badges = await Badge.findAll({
    where: { criteria_type: eventContext.type }
  });

  for (const badge of badges) {
    const alreadyEarned = await UserBadge.exists({
      userId, badgeId: badge.id
    });
    if (alreadyEarned) continue;

    const qualifies = await evaluateCriteria(userId, badge);
    if (qualifies) {
      await UserBadge.create({ userId, badgeId: badge.id });
      await NotificationService.send(userId, {
```

```
        type: "BADGE_EARNED",
        message: `You earned the ${badge.name} badge!`
    });
}
}
```

5.5 Coin Economy

Event	Coins	Notes
Habit completion (Easy)	+1	$\text{floor}(10 \text{ XP} / 10)$
Habit completion (Medium)	+2	$\text{floor}(20 \text{ XP} / 10)$
Habit completion (Hard, no streak)	+3	$\text{floor}(30 \text{ XP} / 10)$
Hard habit + 30-day streak	+4	$\text{floor}(45 \text{ XP} / 10)$
Challenge completion bonus	Varies	Set on challenges.reward_coins
Shop purchase	- cost	Deducted at purchase, stored in purchases.cost_paid

6. Security Documentation

6.1 Authentication Flow

- 14. User submits credentials via HTTPS POST
- 15. Server validates email format and password against bcrypt hash (cost factor 12)
- 16. On success: issue short-lived access token (JWT RS256, 15 min) and refresh token (opaque, 7 days, stored in Redis)
- 17. Refresh token rotation: each use invalidates old token and issues a new pair
- 18. On logout: refresh token deleted from Redis (access token expires naturally)

6.2 Authorization Rules (Row-Level Security)

Resource	Rule
habits	User can only read/write/delete habits where user_id = auth.user_id
habit_logs	Logs are readable by owner only; no edit or delete permitted
user_badges	Read-only for owner; only Badge Service can insert
purchases	Read by owner; write only through Shop Service (verifies coin balance)
friends	Only involved users (user_id or friend_id) can see or modify the record
leaderboards	Public read for global; friends-only filtered by accepted friend list
notifications	Strictly scoped to user_id = auth.user_id; no cross-user access

6.3 Input Validation Requirements

- **3–50 characters, alphanumeric and underscores only, regex: `/^[a-zA-Z0-9_]+$`**/username:
- **RFC 5322 compliant, max 255 characters, normalized to lowercase**email:
- **Minimum 8 characters, must contain uppercase, lowercase, digit, and special character**password:
- **1–100 characters, HTML stripped, no JavaScript injection**habit name:
- **Integer 1–10,000; must be `>= target_value` for full XP**value (log):
- **ISO 8601 format (YYYY-MM-DD); cannot be future-dated**dates:
- **Schema-validated against item type before insert; max 10 KB**JSONB (metadata):

6.4 Data Encryption

- **bcrypt hashed (never stored in plaintext)**Passwords:
- **AES-256 encryption via managed database service (RDS, Supabase)**Database at rest:
- **TLS 1.3 enforced; HSTS headers enabled**Transit:
- **Stored in AWS Secrets Manager or Vault, never in code**Sensitive env vars:
- **4096-bit RSA key pair; private key in HSM/Secrets Manager**JWT secret (RS256):

6.5 Security Headers

```
Content-Security-Policy: default-src 'self'  
X-Frame-Options: DENY  
X-Content-Type-Options: nosniff  
Referrer-Policy: strict-origin-when-cross-origin  
Permissions-Policy: camera=(), microphone=()
```

7. Deployment Documentation

7.1 System Requirements

Component	Minimum	Recommended (Production)
API Server	2 vCPU, 2 GB RAM	4 vCPU, 8 GB RAM (2+ instances)
PostgreSQL	2 vCPU, 4 GB RAM, 20 GB SSD	4 vCPU, 16 GB RAM, 100 GB SSD
Redis	1 vCPU, 512 MB RAM	2 vCPU, 2 GB RAM (HA cluster)
Node.js	v20 LTS	v20 LTS (Docker image: node:20-alpine)

7.2 Environment Variables

Variable	Required	Description
DATABASE_URL	Yes	PostgreSQL connection string with SSL mode
REDIS_URL	Yes	Redis connection URL
JWT_PRIVATE_KEY	Yes	RS256 private key (base64 encoded)
JWT_PUBLIC_KEY	Yes	RS256 public key for token verification
S3_BUCKET	Yes	S3 bucket name for media storage
AWS_ACCESS_KEY_ID	Yes	AWS credentials (prefer IAM roles in production)
PORT	No	API server port (default: 3000)
NODE_ENV	Yes	development staging production
CORS_ORIGIN	Yes	Allowed frontend origin URL
STREAK_RESET_CRON	No	Cron expression (default: "5 0 * * *")

7.3 Docker Configuration

```
# docker-compose.yml
version: "3.9"
services:
  api:
    build: .
    ports: ["3000:3000"]
    environment:
      - DATABASE_URL=${DATABASE_URL}
      - REDIS_URL=redis://redis:6379
```

```
    depends_on: [postgres, redis]
    restart: unless-stopped

    postgres:
      image: postgres:16-alpine
      volumes: [pgdata:/var/lib/postgresql/data]
      environment:
        POSTGRES_DB: habittracker
        POSTGRES_PASSWORD: ${DB_PASSWORD}

    redis:
      image: redis:7-alpine
      volumes: [redisdata:/data]

volumes: { pgdata: {}, redisdata: {} }
```

7.4 Backup Strategy

- Daily automated `pg_dump` to S3 with 30-day retention (encrypted with AES-256)
- Point-in-time recovery (PITR) enabled on managed database with 7-day WAL retention
- Redis RDB snapshots every 6 hours + AOF enabled for persistence
- S3 media bucket cross-region replication to secondary region
- Monthly restore drills to validate backup integrity

7.5 Monitoring

- Prometheus metrics endpoint: `/metrics` (scraped every 15 seconds)
- Key metrics: API latency p50/p95/p99, error rate, active DB connections, Redis hit rate
- Grafana dashboards for gamification KPIs: daily active users, habits logged per hour, XP distributed
- PagerDuty alerts on: error rate > 1%, p99 latency > 2s, DB CPU > 80%

8. Testing Guide

8.1 Unit Tests — Gamification Logic

XP Calculation Tests

```
describe('calculateXP', () => {
  it('returns 10 for easy habit, no streak', () => {
    expect(calculateXP('easy', 0)).toBe(10);
  });
  it('returns 45 for hard habit, 30-day streak', () => {
    expect(calculateXP('hard', 30)).toBe(45);
  });
  it('returns 60 for hard habit, 100-day streak', () => {
    expect(calculateXP('hard', 100)).toBe(60);
  });
});
```

Level Formula Tests

```
describe('calculateLevel', () => {
  it('returns 1 for 0 XP', () => expect(calculateLevel(0)).toBe(1));
  it('returns 2 for 100 XP', () => expect(calculateLevel(100)).toBe(2));
  it('returns 10 for 8100 XP', () => expect(calculateLevel(8100)).toBe(10));
  it('returns 1 for 99 XP', () => expect(calculateLevel(99)).toBe(1));
});
```

Streak Logic Tests

```
describe('StreakTracker', () => {
  it('increments streak on first log of the day', async () => { ... });
  it('does not increment if habit already logged today', async () => { ... });
  it('resets streak if no log yesterday', async () => { ... });
  it('updates max_streak when current exceeds it', async () => { ... });
});
```

8.2 Integration Test Scenarios

Scenario	Assertions
Complete registration flow	User created, JWT returned, welcome notification sent, streak = 0
Log hard habit on day 30	XP += 45, coins += 4, streak = 30, level recalculated, badge checked
Duplicate log same day	HTTP 409 returned, XP unchanged, streak unchanged

Streak reset via cron	User with no yesterday log: current_streak = 0, notification queued
Purchase with insufficient coins	HTTP 402 returned, coins unchanged, no purchase record created
Accept friend request	friends.status = accepted, both users see each other in /friends list
Level-up on habit log	users.level incremented, LEVEL_UP notification sent with new level number

8.3 Performance Test Considerations

- Leaderboard endpoint: target p95 < 100ms with Redis cache; test with 10,000 users in DB
- Habit log endpoint: target p95 < 200ms including badge check; simulate 500 concurrent users
- Streak cron job: must complete processing 100,000 users in < 60 seconds
- Database query: all habit queries must use indexed columns; EXPLAIN ANALYZE all slow queries
- Load test: 1,000 simultaneous habit completions to verify no XP race conditions (use DB transactions)

9. Future Expansions

9.1 Suggested Features

Group Habits

Allow teams to create shared habits with collective completion tracking. Introduces `group_habits` and `group_habit_logs` tables, group membership management, and shared XP pools.

Seasonal Events

Time-limited thematic seasons (e.g., "Summer Challenge Season") with exclusive badges, cosmetics, and challenge sets. A seasons table with start/end dates and associated reward catalog.

AI Habit Recommendations

Integrate an LLM-powered recommendation engine that analyzes user habit history and suggests new habits based on goal alignment, completion patterns, and peer behavior similarity.

Habit Templates & Marketplace

Community-created habit bundles (e.g., "Morning Routine") that users can clone. Includes user-generated content moderation and a rating system.

Mobile Push Notifications

Integrate Firebase Cloud Messaging (FCM) and Apple Push Notification Service (APNs) for streak reminders, friend activity alerts, and challenge deadlines.

9.2 Scalability Considerations

Concern	Strategy
Leaderboards	Redis Sorted Sets (ZADD/ZRANGE) for $O(\log n)$ updates and $O(\log n + k)$ range queries
habit_logs table	Range partition by <code>logged_date</code> (monthly); archive partitions older than 12 months to cold storage
Badge evaluation	Move to async job queue (Bull/BullMQ) to decouple from HTTP request cycle
API throughput	Horizontal pod scaling via Kubernetes HPA; stateless design enables linear scaling
DB reads	Read replicas for leaderboard and stats queries; write to primary only
Session storage	Stateless JWT eliminates session affinity; refresh tokens in Redis cluster

9.3 Potential Microservices Breakdown

As the system grows, the monolithic backend can be decomposed into the following microservices, each independently deployable:

Service	Responsibility
auth-service	Registration, login, token management, password reset
habit-service	CRUD, daily log intake, history queries
gamification-service	XP calculation, level-up events, coin ledger, streak management
badge-service	Criteria evaluation, badge award, user badge inventory
social-service	Friend graph, leaderboards, activity feed
shop-service	Item catalog, purchase validation, inventory management
notification-service	In-app alerts, email digests, push notifications
analytics-service	Aggregated metrics, retention cohorts, XP distribution reports

Communication pattern: Services communicate via an event bus (e.g., Kafka or RabbitMQ) for async events (badge checks, notifications) and REST/gRPC for synchronous calls (auth validation, coin balance checks).

End of Technical Documentation

Gamified Smart Habit Tracker v1.0 | April 2025